

Lab Sheet 07

Title: Aggregation pipeline operators in MongoDB II.

Objective:

- Understanding the Components and Accumulator.

Aggregation Pipeline

The aggregation pipeline consists of stages and each stage transforms the document.

The aggregate () function is used to perform aggregation it can have three operator's stages, expression and accumulator.

```
db.train.aggregate([{$group : { _id :"$id", total : { $sum : "$fare" }}}])
```

Stage Expression Accumulator

Components of Aggregation Pipeline

1.Stages (Operators)

Each stage begins with a MongoDB operator (starts with \$) and performs operations like filtering, grouping, sorting, etc.

Stage Operator	Description
\$match	Filters documents (like SQL WHERE)
\$project	Selects specific fields
\$group	Groups documents and can use accumulators
\$sort	Sorts documents (ascending or descending)
\$skip	Skips n documents
\$limit	Limits to n documents
\$unwind	Breaks down arrays into individual documents
\$out	Writes output to another collection

2.Expressions

These refer to **field names** from documents.

```
{ $group: { _id: "$section", total: { $sum: 1 } } }
```

3.Accumulators

Used within \$group to perform calculations.

Accumulator	Description
\$sum	Adds numeric values
\$count	Counts documents
\$avg	Averages values
\$min	Minimum value
\$max	Maximum value
\$first	First value in group
\$last	Last value in group

Collection: orders

Database: ecommerceDB

OrderID	CustomerName	Product	Category	Quantity	Price	OrderDate
O001	Alice	Phone	Electronics	2	550	2022-01-05
O002	Bob	Laptop	Electronics	1	1200	2022-01-10
O003	Alice	Headphones	Electronics	3	50	2022-01-15
O004	David	T-shirt	Fashion	4	20	2022-01-16
O005	Eva	Shoes	Fashion	1	80	2022-02-01
O006	Frank	Book	Books	5	15	2022-02-10
O007	George	Phone	Electronics	1	550	2022-02-11
O008	Alice	Book	Books	2	15	2022-02-15
O009	Henry	T-shirt	Fashion	2	20	2022-03-01
O010	David	Laptop	Electronics	1	1200	2022-03-02
O011	Alice	Shoes	Fashion	1	80	2022-03-05
O012	Eva	Phone	Electronics	2	550	2022-03-08
O013	Bob	Book	Books	3	15	2022-03-10
O014	Henry	Headphones	Electronics	2	50	2022-03-15
O015	George	Shoes	Fashion	1	80	2022-03-20

```
//Create & Use Database
use ecommerceDB

//Create Collection
db.createCollection("orders")

//Insert Data into orders
db.orders.insertMany([
{ OrderID: "0001", CustomerName: "Alice", Product: "Phone", Category: "Electronics", Quantity:
2, Price: 550, OrderDate: ISODate("2022-01-05") },
{ OrderID: "0002", CustomerName: "Bob", Product: "Laptop", Category: "Electronics", Quantity:
1, Price: 1200, OrderDate: ISODate("2022-01-10") },
{ OrderID: "0003", CustomerName: "Alice", Product: "Headphones", Category: "Electronics",
Quantity: 3, Price: 50, OrderDate: ISODate("2022-01-15") },
{ OrderID: "0004", CustomerName: "David", Product: "T-shirt", Category: "Fashion", Quantity: 4,
Price: 20, OrderDate: ISODate("2022-01-16") },
{ OrderID: "0005", CustomerName: "Eva", Product: "Shoes", Category: "Fashion", Quantity: 1,
Price: 80, OrderDate: ISODate("2022-02-01") },
{ OrderID: "0006", CustomerName: "Frank", Product: "Book", Category: "Books", Quantity: 5,
Price: 15, OrderDate: ISODate("2022-02-10") },
{ OrderID: "0007", CustomerName: "George", Product: "Phone", Category: "Electronics", Quantity:
1, Price: 550, OrderDate: ISODate("2022-02-11") },
{ OrderID: "0008", CustomerName: "Alice", Product: "Book", Category: "Books", Quantity: 2,
Price: 15, OrderDate: ISODate("2022-02-15") },
{ OrderID: "0009", CustomerName: "Henry", Product: "T-shirt", Category: "Fashion", Quantity: 2,
Price: 20, OrderDate: ISODate("2022-03-01") },
{ OrderID: "0010", CustomerName: "David", Product: "Laptop", Category: "Electronics", Quantity:
1, Price: 1200, OrderDate: ISODate("2022-03-02") },
{ OrderID: "0011", CustomerName: "Alice", Product: "Shoes", Category: "Fashion", Quantity: 1,
Price: 80, OrderDate: ISODate("2022-03-05") },
{ OrderID: "0012", CustomerName: "Eva", Product: "Phone", Category: "Electronics", Quantity: 2,
Price: 550, OrderDate: ISODate("2022-03-08") },
{ OrderID: "0013", CustomerName: "Bob", Product: "Book", Category: "Books", Quantity: 3, Price:
15, OrderDate: ISODate("2022-03-10") },
{ OrderID: "0014", CustomerName: "Henry", Product: "Headphones", Category: "Electronics",
Quantity: 2, Price: 50, OrderDate: ISODate("2022-03-15") },
{ OrderID: "0015", CustomerName: "George", Product: "Shoes", Category: "Fashion", Quantity: 1,
Price: 80, OrderDate: ISODate("2022-03-20") }
])
```

1: Find all orders in the "Electronics" category

```
db.orders.aggregate([
  {
    $match: { Category: "Electronics" }
  }
])
```

2: Calculate total revenue (Quantity × Price) from Electronics category

```
db.orders.aggregate([
  {
    $match: { Category: "Electronics" }
  },
  {
    $group: {
      _id: null,
      totalRevenue: {
        $sum: { $multiply: ["$Quantity", "$Price"] }
      }
    }
  }
])
```

3: Group by Category and calculate total revenue per category

```
db.orders.aggregate([
  {
    $group: {
      _id: "$Category",
      totalRevenue: {
        $sum: { $multiply: ["$Quantity", "$Price"] }
      }
    }
  }
])
```

4: Sort categories by total revenue in descending order

```
db.orders.aggregate([
  {
    $group: {
      _id: "$Category",
      totalRevenue: {
        $sum: { $multiply: ["$Quantity", "$Price"] }
      }
    }
  },
  {
    $sort: -totalRevenue
  }
])
```

```

    }
  }
},
{
  $sort: { totalRevenue: -1 }
}
])

```

5: Find the highest quantity ordered for any product

```

db.orders.aggregate([
  {
    $group: {
      _id: null,
      maxQuantity: { $max: "$Quantity" }
    }
  }
])

```

6: Display total quantity of each product ordered by each customer

```

db.orders.aggregate([
  {
    $group: {
      _id: {
        customer: "$CustomerName",
        product: "$Product"
      },
      totalQuantity: { $sum: "$Quantity" }
    }
  },
  {
    $sort: {
      "_id.customer": 1,
      "_id.product": 1
    }
  }
])

```

7: Find orders placed between 2022-02-01 and 2022-03-01

```
db.orders.aggregate([
  {
    $match: {
      OrderDate: {
        $gte: "2022-02-01",
        $lte: "2022-03-01"
      }
    }
  }
])
```

If stored as **ISODate**, use this:

```
db.orders.aggregate([
  {
    $match: {
      OrderDate: {
        $gte: ISODate("2022-02-01"),
        $lte: ISODate("2022-03-01")
      }
    }
  }
])
```

8: Unwind Product Categories if stored as array (hypothetical) (Assuming Category: ["Fashion", "Men"])

```
db.orders.aggregate([
  {
    $unwind: "$Category"
  }
])
```

9: Write filtered data into a new collection

```
db.orders.aggregate([
  {
    $match: { Category: "Electronics" }
  },
  {
    $out: "electronics_orders"
  }
])
```

```
//Bonus (Optional - Good for Exams)  
db.orders.aggregate([  
  {  
    $group: {  
      _id: "$Category",  
      avgPrice: { $avg: "$Price" }  
    }  
  }  
])
```